

```
boolean (boolean)
data = True
data : ", data_boolean
ta khusus (complex)
ex = complex(5,6)
: ", data_complex,
anakan tipe data dari
import c_double
c_double(10.5)
data
```

Visualización de Datos con Python

De los datos crudos al conocimiento accionable: un módulo completo para estudiantes de análisis de datos.

 MÓDULO DE VISUALIZACIÓN

¿Qué es la visualización de datos?

La visualización transforma datos crudos en conocimiento comprensible

Python no solo sirve para programar: permite **analizar, representar y comunicar** información de forma poderosa y flexible.



Descubrir patrones

Identifica tendencias, relaciones y comportamientos ocultos en los datos.



Detectar anomalías

Encuentra valores atípicos que pueden indicar errores u oportunidades.



Tomar decisiones

Una buena visualización facilita decisiones claras y accionables, no solo se ve atractiva.

¿Por qué visualizar datos?

Las tablas y los números suelen **ocultar información crítica**. Los gráficos la revelan de inmediato.



Tendencias

Permite encontrar tendencias a lo largo del tiempo de un solo vistazo.



Comparaciones

Facilita comparar categorías o grupos con claridad y precisión.



Outliers

Ayuda a detectar comportamientos inusuales o datos atípicos rápidamente.



Comunicación

Mejora la comunicación con audiencias técnicas y no técnicas por igual.



Elegir el gráfico correcto

Cada tipo de gráfico responde a una **pregunta diferente**. La elección incorrecta puede confundir o distorsionar el mensaje.

Necesidad	Visualización recomendada
Comparar categorías	Gráfico de barras
Mostrar evolución temporal	Gráfico de líneas
Analizar relaciones entre variables	Gráfico de dispersión
Ver distribuciones	Histograma o diagrama de caja
Mostrar proporciones	Gráfico circular
Ver intensidades o correlaciones	Mapa de calor

Matplotlib: la base de la visualización

Matplotlib es la biblioteca **más fundamental y flexible** de Python para crear gráficos estáticos de alta calidad. Es ideal para reportes, notebooks científicos y documentación.

`plt.plot()`

Gráficos de líneas para series temporales y tendencias.

`plt.bar()` / `plt.hist()`

Barras y distribuciones para comparar frecuencias.

`plt.scatter()`

Dispersión para explorar relaciones entre variables.



Seaborn: visualización estadística

Seaborn **complementa a Matplotlib** con gráficos estadísticos más expresivos y un diseño atractivo por defecto. Simplifica el trabajo con datasets de pandas.



Distribuciones

Histogramas y KDE plots para visualizar la distribución de variables continuas.



Heatmaps

Matrices de correlación y valores de intensidad entre variables.



Boxplots

Visualiza dispersión, mediana y outliers de forma compacta y expresiva.



Relaciones

Gráficos de relación entre variables con agrupación por categorías.

Plotly: gráficos interactivos

Plotly transforma los gráficos estáticos en **experiencias explorables**. Con Plotly Express, puedes crear visualizaciones profesionales con pocas líneas de código.

Zoom & Pan

Incorpora zoom, pan, hover y selección de datos de forma nativa.

Series temporales

Range slider para explorar intervalos de tiempo con precisión.

Mapas

Mapas coropléticos para visualizar datos geoespaciales.

Dashboards

Paneles con filtros, KPIs e interactividad sin necesidad de backend.



Bokeh y Dash: web e interactividad avanzada

Bokeh: visualizaciones web

Bokeh se enfoca en gráficos interactivos **listos para la web**. Trabaja con Figure, glifos y ColumnDataSource. Puede exportar gráficos como archivos HTML independientes con zoom, hover y pan nativos.

- Buen rendimiento con grandes volúmenes de datos
- Layouts con múltiples gráficos coordinados
- Personalización total de colores y herramientas

Dash: aplicaciones analíticas

Dash permite construir **aplicaciones web analíticas completas** usando solo Python. Su sistema de callbacks conecta entradas del usuario con salidas visuales en tiempo real.

app.layout

Estructura visible de la aplicación.

Input / Output

Componentes que disparan y reciben actualizaciones.

State

Valor leído sin disparar el callback automáticamente.

¿Cuál herramienta elegir?

No existe una única herramienta mejor. La elección depende del **objetivo**, la **audiencia** y el **nivel de interactividad** requerido.

Herramienta	Mejor uso	Ventaja principal
Matplotlib	Gráficos estáticos y publicaciones	Máximo control y alta calidad
Seaborn	Análisis estadístico exploratorio	Visualizaciones claras con poco código
Plotly	Exploración interactiva	Zoom, hover, filtros y dashboards
Bokeh	Visualizaciones web interactivas	HTML, layouts y rendimiento
Dash	Aplicaciones analíticas	Dashboards web con callbacks
Power BI / Superset	Business Intelligence empresarial	Paneles y monitoreo sin código



Data Storytelling

El Data Storytelling combina **datos**, **visualizaciones** y **narrativa** para transformar hallazgos en mensajes que generan acción.

01

Presentar el contexto

¿Cuál es la situación? ¿Qué pregunta queremos responder?

02

Explorar los datos

¿Qué ocurrió? Muestra los hechos con evidencia visual sólida.

03

Mostrar hallazgos

¿Por qué ocurrió? Destaca los patrones más relevantes.

04

Implicancias y decisiones

¿Qué significa? ¿Qué acciones se pueden tomar a partir de esto?

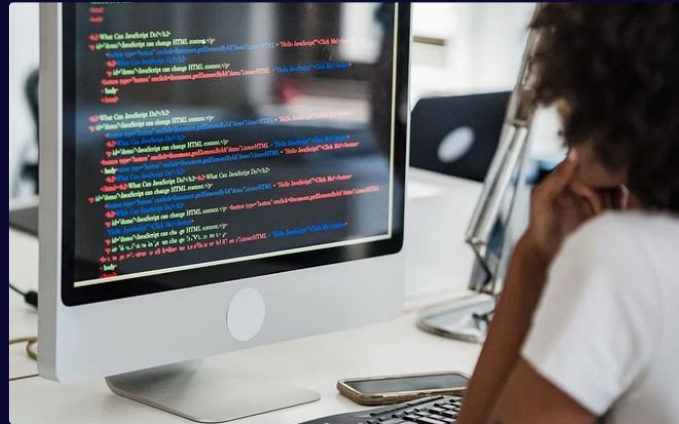
Diseño de presentaciones efectivas

Una presentación basada en datos debe **adaptarse a su audiencia**. El mismo análisis se comunica de forma diferente según quién lo reciba.



Audiencia Ejecutiva

KPIs, tendencias clave, conclusiones y recomendaciones directas. Sin tecnicismos.



Audiencia Técnica

Metodología, detalle estadístico, código y análisis profundo de los datos.



Audiencia General

Lenguaje claro, gráficos simples, pocas métricas por diapositiva y analogías accesibles.

Carga cognitiva en visualizaciones

¿Qué es la carga cognitiva?

Es el **esfuerzo mental necesario** para comprender una visualización. Una visualización efectiva lo **minimiza**, permitiendo que el mensaje llegue sin fricciones.

- ❏ La mejor visualización no es la que muestra más información, sino la que permite entender el mensaje con **menos esfuerzo**.

Cómo reducirla

- **Eliminar ruido visual**
Evitar sombras, 3D, bordes excesivos y colores sin propósito.
- **Usar jerarquía visual**
Dirigir la atención con tamaño, contraste y posición.
- **Dividir en bloques**
Mostrar información compleja de forma progresiva y ordenada.

Sesgos y errores comunes

Un análisis visual puede ser **incorrecto** si los datos o la interpretación están sesgados, incluso cuando los gráficos se ven perfectos.

Sesgos frecuentes

Muestra
insuficiente

Sesgo de
supervivencia

Sesgo de muestreo

Sesgo de
confirmación

Sesgo de anclaje

Errores habituales

Confundir
correlación con
causalidad

Ignorar valores
atípicos

Eliminar datos sin
evaluar impacto

Manipular escalas
de los ejes

Usar gráficos que exageran diferencias

Flujo recomendado de trabajo

Un proyecto de visualización sólido debe seguir un **proceso claro y reproducible**, desde los datos en bruto hasta la decisión final.



Cada etapa es crucial: saltar pasos puede producir visualizaciones incorrectas o decisiones basadas en análisis deficientes.

CONCLUSIÓN

Visualizar datos es comunicar evidencia

Técnica

Matplotlib y Seaborn para explorar y explicar datos con precisión y calidad.

Interactividad

Plotly, Bokeh y Dash agregan exploración dinámica y dashboards en tiempo real.

Comunicación

El diseño visual y el storytelling convierten hallazgos en decisiones concretas.

Integridad

Detectar sesgos y evitar errores protege la calidad y honestidad del análisis.

Visualizar datos no consiste solo en crear gráficos: consiste en **comunicar evidencia de manera clara, honesta y útil.**